Build Guide: A FastStone-Style Screen Capture and Recorder App

A complete specification and step-by-step build process that any AI coding assistant (or human developer) can follow to build a lightweight Windows screen capture, annotation and screen recording tool with the same feature set as the product you described.

Working name: SnapForge (use your own original name, logo and icons; do not copy FastStone's name, branding or assets).

---

1. How to use this guide with an AI

1. Give the AI **Section 2 (Decisions)** and **Section 3 (Architecture)** first, as permanent context.
2. Then feed it **one phase at a time** from Section 6. Never ask for the whole app in one prompt.
3. After each phase, run the **acceptance tests** listed for that phase before moving on.
4. Keep the AI inside the folder structure in Section 4 so code stays consistent between sessions.
5. Use the prompt templates in Section 9 for every request.

Rule of thumb: small, testable steps. A working capture tool after Phase 2 is better than a half-finished everything-tool.

---

2. Key decisions (fixed so the AI does not drift)

Topic	Decision
Platform	Windows 10/11, 64-bit (the original is Windows-only because it relies on Win32 APIs)
Language	C# 12 on .NET 8
UI framework	WPF for the editor and panels; a thin Win32 layer for overlays and hotkeys

Topic	Decision
Capture APIs	GDI (BitBlt/PrintWindow) for stills; Windows.Graphics.Capture or DXGI Desktop Duplication for video
Object/menu detection	UI Automation (System.Windows.Automation) plus Win32 EnumChildWindows
Hotkeys	RegisterHotKey (global)
OCR	Windows.Media.Ocr (built in) with optional Tesseract fallback
Video encoding	FFmpeg (bundled ffmpeg.exe, piped raw frames) or Media Foundation via SharpDX/Vortice
Audio	NAudio: WasapiCapture (microphone) and WasapiLoopbackCapture (speakers)
Image encoding	System.Windows.Media.Imaging encoders plus SixLabors.ImageSharp for TGA/PCX/WEBP
Settings storage	JSON file in %AppData%\SnapForge\settings.json
Installer	WiX or Inno Setup; also offer a portable ZIP
Target size	Under 15 MB installed, under 80 MB RAM idle

If you prefer cross-platform, swap in Avalonia UI and platform-specific capture backends, but expect much more work. Windows-first is the realistic path.

---

3. Architecture

Separate the app into independent modules that talk through interfaces. This is what lets an AI build and test each part alone.

```
UI layer (WPF)
  CapturePanel  |  EditorWindow  |  SettingsWindow  |  RecorderWindow  |
Overlays
        |
Application services
  CaptureService   OutputService   HotkeyService   SettingsService
  RecorderService  OcrService      ToolsService (picker, magnifier, ruler,
crosshair)
        |
Core libraries
  Capture.Engines   Imaging   Annotations   Encoding   Native (P/Invoke)
```

Core interfaces

```
public interface ICaptureEngine {
    CaptureMode Mode { get; }
    Task<CaptureResult> CaptureAsync(CaptureOptions options,
CancellationToken ct);
}

public interface IOutputTarget {
    string Name { get; }              // Editor, File, Clipboard, Printer,
Email, Office, FTP
    Task SendAsync(CaptureResult result, OutputContext ctx);
}

public interface IAnnotation {          // every drawable object in the
editor
    Rect Bounds { get; set; }
    void Render(DrawingContext dc);
    bool HitTest(Point p);
    AnnotationDto ToDto();             // for FSC save format
}
```

`CaptureResult` holds a `BitmapSource`, source window title, timestamp, monitor DPI and capture mode. Every capture mode returns the same type, so every output target works with every mode.

---

4. Folder structure

```
SnapForge/
  SnapForge.sln
  src/
    SnapForge.App/            WPF startup, tray icon, DI container
    SnapForge.Native/         P/Invoke: user32, gdi32, dwmapi, shcore,
hotkeys
    SnapForge.Capture/        engines: Window, Object, Menu, Fullscreen,
Region,
                              Freehand, FixedRegion, Scrolling, AutoRepeat
    SnapForge.Imaging/        resize, crop, rotate, sharpen, brightness,
effects
    SnapForge.Annotations/    shapes, text, arrows, highlight, callout, step
number,
                              emoji, watermark, blur, spotlight, FSC
serializer
    SnapForge.Editor/         tabbed editor, workspaces, toolbars, undo/redo
    SnapForge.Output/         file, clipboard, printer, email, Office, FTP,
PDF
    SnapForge.Recorder/       screen, audio, webcam, cursor, encoder, timer,
video editor
    SnapForge.Tools/          color picker, magnifier, crosshair, ruler,
focus, pin-to-screen
    SnapForge.Ocr/            OCR service
    SnapForge.Settings/       settings model, hotkey map, filename templates
  tests/
    SnapForge.Tests/          unit tests per module
  assets/                     icons, ffmpeg.exe
  docs/
```

---

5. Feature checklist mapped to modules

Use this as the master to-do list. Tick items off phase by phase.

Capture (SnapForge.Capture)

- [ ] Active window, any window (hover-highlight selection)
- [ ] Window object / child control (UI Automation hover detection)
- [ ] Menu capture including multi-level menus
- [ ] Multiple windows/objects in one capture

- [ ] Full screen, all monitors, single monitor
- [ ] Rectangular region, freehand region, fixed-size region
- [ ] Scrolling window / web page (stitching)
- [ ] Auto-capture at intervals
- [ ] Capture text from image (OCR)
- [ ] Multi-monitor and high-DPI/4K correctness

Output (SnapForge.Output)

- [ ] Editor, file, clipboard, printer, email
- [ ] Word / Excel / PowerPoint / OneNote insertion
- [ ] FTP upload
- [ ] Automatic filename generation with template; auto-save without dialog

Editor (SnapForge.Editor and Annotations)

- [ ] Tabs and multiple workspaces
- [ ] Annotations: text (straight/curved), arrows, highlight, rectangle, circle, callout, step numbers, emoji, watermark
- [ ] Blur selected area, spotlight, drop-shadow, frame, torn edge, fade edge
- [ ] Caption, resize, crop, rotate, sharpen, brighten, color adjust
- [ ] Undo/redo, external editor support
- [ ] Save as BMP, GIF, JPEG, PCX, PNG, TGA, TIFF, WEBP, PDF and the lossless project format (.snf)
- [ ] Scanner import, combine images, images to one PDF, batch print

Recorder (SnapForge.Recorder)

- [ ] Region/window/full-screen recording
- [ ] Microphone and speaker audio, webcam overlay
- [ ] Mouse movement and click highlighting
- [ ] MP4 and WMV output, timer scheduling
- [ ] Built-in video editor: annotate, zoom effects, cut sections
- [ ] Convert to GIF; Convert/resize MP4 tool

Utilities (SnapForge.Tools)

- [ ] Color picker, magnifier, crosshair, ruler, screen focus
- [ ] Pin screenshot on top (sticky note)

- [ ] Full-screen viewer with drag and drop onto the panel

Platform

- [ ] Global hotkeys, tray icon, run at startup, small memory footprint
- [ ] Touch support (tap, swipe, pinch)

---

6. Build phases (the actual making process)

Phase 0: Setup (half a day)

1. Create the solution and projects from Section 4 with `dotnet new`.
2. Add NuGet packages: `NAudio`, `SixLabors.ImageSharp`, `CommunityToolkit.Mvvm`, `Microsoft.Extensions.DependencyInjection`, `Hardcodet.NotifyIcon.Wpf`, `xunit`.
3. Create `SnapForge.Native` with P/Invoke wrappers: `GetForegroundWindow`, `GetWindowRect`, `DwmGetWindowAttribute(DWMWA_EXTENDED_FRAME_BOUNDS)`, `BitBlt`, `PrintWindow`, `RegisterHotKey`, `GetCursorPos`, `EnumWindows`, `GetDpiForMonitor`.
4. Set the app manifest to **PerMonitorV2 DPI awareness**. Skipping this causes wrong region sizes on 4K and mixed-DPI monitors, the most common bug in capture tools.

Acceptance: solution builds, tray icon appears, app starts under 50 MB RAM.

Phase 1: Capture Panel, hotkeys, fullscreen and region capture

1. Build a small always-on-top **Capture Panel** (a toolbar of icon buttons: Active window, Window/Object, Rectangle, Freehand, Fixed region, Scrolling window, Full screen, Menu, Screen recorder, Tools, Settings).
2. `HotkeyService`: register defaults, for example PrintScreen = full screen, Ctrl+Shift+R = region, Ctrl+Shift+W = active window. Let users rebind in Settings. Handle conflicts by showing a message.
3. `FullscreenEngine`: for each monitor, `BitBlt` from the screen DC into a bitmap using the monitor's real pixel bounds; stitch for "all monitors".
4. `RegionEngine`: show a borderless, topmost, semi-transparent overlay over the whole virtual desktop containing a frozen screenshot. User drags a rectangle; show live size and a magnified crosshair. Esc cancels. Return the cropped pixels from the frozen image (this avoids capturing the overlay itself).
5. `FixedRegionEngine`: same overlay but the rectangle is a preset size that follows the mouse; click to capture.

6. `FreehandEngine`: user draws a path; create a mask from the polygon and make pixels outside transparent.

Acceptance: each hotkey produces a correct image on single and dual monitor setups at 100% and 150% scaling; image saved to clipboard.

Phase 2: Window, object, menu and scrolling capture

1. `WindowEngine`: on hover use `WindowFromPoint` + `GetAncestor(GA_ROOT)`; draw a highlight border. On click, capture with `PrintWindow(PW_RENDERFULLCONTENT)`; fall back to `BitBlt` of the DWM frame bounds. Handle windows partly off-screen and minimized windows (restore briefly).
2. `ObjectEngine`: use UI Automation `AutomationElement.FromPoint` to find the control under the cursor, and show its bounding rectangle. Allow the mouse wheel to go up/down the parent chain.
3. `MenuEngine`: menus are separate topmost windows with class `#32768`. On the hotkey, wait for the user to open the menu, then capture the frozen desktop and crop to the menu window rectangles (union them to support submenus). Add a 1-2 second delay option.
4. `MultiCapture`: allow holding Ctrl to add several windows/objects, then compose them in one image on a transparent or chosen background.
5. `ScrollingEngine`: this is the hardest part. Algorithm:
   - Identify the scrollable target (browser, document, or any window under cursor).
   - Capture the viewport, send scroll input (mouse wheel or PageDown via `SendInput`), wait 150-300 ms for render, capture again.
   - Find the overlap between consecutive frames by comparing the bottom rows of frame N with rows in frame N+1 (row hashing or normalized cross-correlation). Append only the new rows.
   - Stop when two consecutive frames are identical (end of page) or a max height is reached.
   - Ignore sticky headers/footers by detecting rows identical across all frames and trimming them.
6. `AutoRepeatEngine`: timer that runs any chosen engine every N seconds, saving files with the filename template.

Acceptance: captures a long web page into one seamless image; captures a context menu with a submenu; hover highlights are accurate.

Phase 3: Output system and filename templates

1. Implement `IOutputTarget` for: Editor, File, Clipboard, Printer (`PrintDialog` / `PrintDocument`), Email (MAPI or `mailto:` with saved attachment path), FTP (`FtpWebRequest` or FluentFTP).
2. Office targets: use COM interop (late binding with `dynamic`) to paste into the active Word, Excel, PowerPoint or OneNote document, so no Office dependency is required to run the app.
3. **Filename template** engine with tokens: `{yyyy}{MM}{dd}-{HH}{mm}{ss}`, `{counter:000}`, `{window}`, `{app}`, `{mode}`. Example: `Capture_{counter:000}_{window}`. Sanitize illegal characters.
4. **Automatic file generation:** when output is File, save straight to the output folder and format without showing Save As.
5. Settings screen for output destination, folder, format, JPEG quality, and a toggle to also copy to clipboard.

Acceptance: capture to each target works; two quick captures never overwrite each other.

Phase 4: The editor

1. Tabbed `EditorWindow`. Each tab holds an `EditorDocument` (base image + ordered list of `IAnnotation` + undo stack). Workspaces are named groups of tabs, each remembering its last folder.
2. Canvas: WPF `Canvas` with zoom/pan, using a `DrawingVisual` per annotation for speed.
3. Command pattern for undo/redo: every edit is an `IEditCommand` with `Do()` and `Undo()`.
4. Annotation tools: selection with 8 resize handles, then Text, Arrow, Line, Curved line, Rectangle, Ellipse, Highlight (multiply blend), Callout, Step number (auto-incrementing circle), Emoji (Segoe UI Emoji), Watermark (text or image with opacity), Blur/Pixelate region.
5. Text on a curve: place glyphs along a Bezier path with `FormattedText.BuildGeometry` and rotate each glyph by the path tangent.
6. Image operations (in `SnapForge.Imaging`, operating on `WriteableBitmap`): crop, resize (Lanczos or bicubic), rotate/flip, sharpen (unsharp mask), brightness/contrast/gamma, color balance.

7. Effects: drop-shadow (Gaussian-blurred alpha offset), frame/border, torn edge (randomized zigzag alpha mask), fade edge (gradient alpha mask), spotlight (darken all but the selected region), caption (extra strip with text).
8. Pin to screen: borderless topmost window showing the image, draggable, with right-click close and opacity.
9. Drag and drop images from Explorer/browser onto the Capture Panel to open a full-screen viewer.

Acceptance: 100-step undo works; every effect is non-destructive until flattened on save.

Phase 5: File formats and project format

1. Standard formats: PNG, JPEG, BMP, GIF, TIFF via WPF encoders; WEBP, TGA, PCX via ImageSharp (add a small PCX writer if no library).
2. PDF: write one page per image using PdfSharp or QuestPDF; implement "convert multiple images into a single PDF".
3. **Project format (.snf):** a ZIP containing `image.png`, `annotations.json` (list of AnnotationDto with type, bounds, style, text) and `meta.json`. Loading rebuilds editable objects, which is what makes it lossless and re-editable.
4. Tools: combine images (horizontal/vertical/grid), batch print, scanner import via WIA (`WIA.CommonDialog`).

Acceptance: save .snf, reopen, and every annotation is still editable.

Phase 6: Screen recorder

1. **Video source:** use `Windows.Graphics.Capture` (Windows 10 1903+) for window or monitor capture with GPU frames; fall back to DXGI Desktop Duplication. Target 30 fps (allow 15/24/30/60).
2. **Cursor and clicks:** draw the cursor into each frame yourself (GetCursorInfo + DrawIconEx), plus a highlight circle and click ripple using a low-level mouse hook.
3. **Audio:** NAudio `WasapiCapture` for the microphone and `WasapiLoopbackCapture` for system sound; mix with `MixingSampleProvider`; keep audio timestamps aligned to the video clock (use one `Stopwatch` as master clock).
4. **Webcam:** capture with MediaCapture/AForge/OpenCvSharp and composite as a picture-in-picture overlay in a chosen corner.

5. **Encoding:** pipe raw BGRA frames into `ffmpeg.exe` (`-f rawvideo -pix_fmt bgra -s WxH -r FPS -i - ... -c:v libx264 -crf 23 -preset veryfast`) with the audio as a second input; output MP4 (H.264/AAC). For WMV use `-c:v wmv2 -c:a wmav2`. Check the license of any bundled FFmpeg build (use an LGPL build and credit it).
6. **Timer scheduling:** start at a set time or stop after a duration.
7. **Recorder UI:** small control bar with Start/Pause/Stop, region selector, source toggles (mic, speakers, webcam, cursor), and a countdown.
8. **Video editor:** timeline with a thumbnail strip; tools to cut a section, add text/arrow/shape annotations for a time range, and add zoom effects (keyframed crop rectangles). Export by re-encoding through FFmpeg filters (`drawtext`, `overlay`, `crop` + `scale`, `trim`/`concat`).
9. **Convert to GIF** (FFmpeg palettegen/paletteuse) and a **Convert to MP4** tool with resolution and quality presets.

Acceptance: a 10-minute recording with mic + speakers stays in sync (drift under 100 ms), file plays in common players, and CPU stays under 25% at 1080p30.

Phase 7: Utility tools

1. **Color picker:** magnified live view under the cursor, click to copy HEX/RGB/HSL; keep a history of recent colors.
2. **Screen magnifier:** a topmost lens window following the cursor, zoom 2x-16x.
3. **Screen crosshair:** full-screen transparent click-through window drawing lines at the cursor.
4. **Screen ruler:** draggable transparent ruler in px, with horizontal/vertical toggle and rotation.
5. **Screen focus:** dim everything except an area around the cursor (useful for presentations).
6. **OCR (Capture Text):** select a region, run `Windows.Media.Ocr.OcrEngine.TryCreateFromUserProfileLanguages()`, copy text to clipboard, show it in a small result window. Allow choosing the language.

Acceptance: each tool starts from a hotkey and can be closed with Esc.

Phase 8: Polish, performance, touch and packaging

1. Touch support: handle `Manipulation` events in the editor (tap select, swipe pan, pinch zoom) and larger hit targets on the panel.

2. Run at Windows startup (HKCU Run key), minimize to tray, single-instance mutex.
3. Memory: dispose bitmaps, use `Freeze()` on WPF images, avoid keeping every capture in memory; cap the undo stack by size.
4. Crash safety: wrap services with logging to `%AppData%\\SnapForge\\logs`; autosave editor tabs every 60 seconds.
5. Localization-ready resources (`.resx`), keyboard shortcuts in all menus.
6. Build installer (Inno Setup/WiX) and a portable ZIP; code-sign if possible to avoid SmartScreen warnings.

---

7. Data models (give these to the AI verbatim)

```
public enum CaptureMode { ActiveWindow, Window, Object, Menu, Fullscreen,
    Rectangle, Freehand, FixedRegion, Scrolling, AutoRepeat }

public record CaptureOptions(CaptureMode Mode, int DelayMs, bool
IncludeCursor,
    Size? FixedSize, int MaxScrollHeight);

public record CaptureResult(BitmapSource Image, string? WindowTitle,
    string? AppName, DateTime Timestamp, double DpiX, double DpiY,
CaptureMode Mode);

public record AnnotationDto(string Type, Rect Bounds, string? Text,
    string StrokeColor, string FillColor, double StrokeWidth,
    Dictionary<string,string> Extra);

public class AppSettings {
    public OutputKind DefaultOutput { get; set; }
    public string OutputFolder { get; set; }
    public string OutputFormat { get; set; } = "png";
    public string FilenameTemplate { get; set; } = "Capture_{yyyy}{MM}{dd}-
{HH}{mm}{ss}";
    public bool AutoSaveWithoutDialog { get; set; }
    public Dictionary<string,string> Hotkeys { get; set; }
    public bool RunAtStartup { get; set; }
    public RecorderSettings Recorder { get; set; }
}
```

---

8. Testing strategy

- **Unit tests:** filename template, scroll-stitch overlap finder (use synthetic images), annotation serialization round-trip, undo/redo, image filters (compare to known outputs).
- **Manual matrix:** 100%, 125%, 150%, 200% scaling; one and two monitors; mixed DPI; Windows 10 and 11; light and dark themes.
- **Capture fixtures:** keep a folder of reference screenshots and pixel-compare to catch regressions.
- **Recorder tests:** record a screen with a stopwatch playing and a beep tone; verify sync by eye and by waveform.
- **Stress:** 500 captures in a row, memory must return to baseline.

---

9. Prompt templates for any AI

Context prompt (paste once per session):

> You are a senior Windows desktop engineer. We are building SnapForge, a lightweight screen capture, annotation and recording app in C# 12, .NET 8, WPF. Follow the folder structure, interfaces and data models I paste below exactly. Write complete, compilable files, not snippets. Use PerMonitorV2 DPI-correct coordinates everywhere. Do not add features not requested. After the code, list how to test it.

Phase prompt:

> Implement Phase [N], step [X] from the plan: [paste the step]. Create or modify only these files: [list]. Include unit tests where the plan calls for them. Explain any Win32 API you use in one line each.

Bug-fix prompt:

> Here is the error/behavior: [paste]. Here is the relevant code: [paste]. Find the root cause first, then give the minimal fix. Do not rewrite unrelated code.

Review prompt:

> Review this module for DPI bugs, GDI handle leaks (every GetDC needs ReleaseDC, every CreateCompatibleBitmap needs DeleteObject), threading problems (UI work on the dispatcher only), and memory leaks.

---

10. Common pitfalls and how to avoid them

Problem	Fix
Wrong size/offset on 4K or mixed-DPI screens	PerMonitorV2 manifest; convert all coordinates to physical pixels
Overlay appears in its own screenshot	Capture first, freeze, then show overlay over the frozen image
Black image for some windows (browsers, games)	Use `PrintWindow` with `PW_RENDERFULLCONTENT` or Windows.Graphics.Capture
Window capture includes invisible shadow border	Use `DWMWA_EXTENDED_FRAME_BOUNDS` for the real rectangle
GDI handle leaks crash app after many captures	Wrap handles in `SafeHandle` classes, test with Task Manager's GDI column
Scroll stitching duplicates rows	Trim sticky headers; require a minimum overlap match score
Audio drifts from video	One master clock; resample audio, drop or duplicate video frames to keep constant fps
Hotkey does not register	Another app owns it; detect failure from `RegisterHotKey` and prompt for another
Huge RAM from undo stack	Store diffs or small commands; cap by megabytes
Large video files	CRF 23-28, H.264, 30 fps, variable bitrate

---

11. Legal and licensing notes

- Build your own original app. Do not copy FastStone's name, icons, UI artwork, installer text or code. Reimplementing the *ideas* (a capture panel, scrolling capture, annotation tools) is fine; copying assets is not.
- Check licenses of dependencies: FFmpeg (LGPL/GPL depending on build), NAudio (MIT), ImageSharp (split license, check commercial terms), Tesseract (Apache 2.0).
- Provide a privacy statement: the app should store captures locally and never upload without an explicit user action (FTP/email).

---

12. Suggested timeline (one developer with AI help)

Phase	Scope	Estimate
0	Setup and native layer	1 day
1	Panel, hotkeys, fullscreen and region	3-4 days
2	Window, object, menu, scrolling	5-7 days
3	Outputs and filenames	3 days
4	Editor, annotations, effects	10-14 days
5	File formats and project format	3 days
6	Recorder and video editor	10-14 days
7	Utility tools and OCR	4-5 days
8	Polish and packaging	4-5 days

A **minimum viable product** is Phases 0-4 (about 4 weeks): a capture panel, all main capture modes, outputs, and a good editor. Add the recorder after that.

---

13. Quick-start: the first three prompts to send

1. "Using the folder structure and interfaces below, generate the solution skeleton, `.csproj` files, DI setup, tray icon, and the `SnapForge.Native` P/Invoke wrappers listed in Phase 0."

2. "Implement `HotkeyService` and the `FullscreenEngine` with multi-monitor and PerMonitorV2 support, plus a Capture Panel with a Full screen button that copies to clipboard."
3. "Implement the frozen-overlay `RegionEngine` with live size readout and a magnifier loupe, returning a `CaptureResult`."

From there, continue phase by phase, testing each step before moving on.

---

14. Alternatives if Windows-only C# is not what you want

- **Python prototype:** PyQt6 + `mss` (capture) + `Pillow` + `pynput` + `opencv-python` + `ffmpeg`. Faster to write, larger and slower than native, weaker for object/menu capture.
- **Electron/Tauri:** use `desktopCapturer` or native Rust crates (`xcap`, `scrap`); good for cross-platform, but menu capture and scrolling capture are harder.
- **macOS/Linux:** replace GDI with ScreenCaptureKit (macOS) or PipeWire/X11 (Linux), and Win32 hotkeys with platform equivalents.

If you tell me which stack you want, I can expand any single phase into full, ready-to-compile code for you to hand to your AI tool.